TECNOLOGÍAS PARA LA AUTOMATIZACIÓN · 4º AÑO

3º Parcial · IoT

Resumen de la ESP32 y la Internet de las cosas + los sketches que tenés que saber leer

Cuestionario en el campus | Opción múltiple · casi siempre varias correctas

Fuente: Apunte IoT 2026 y clases del profe

Cómo es el parcial (según los de 2025)

10 preguntas de opción múltiple de 10 puntos cada una (nota sobre 100: 64 → un 6). En casi todas hay **más de una respuesta correcta** y el puntaje es parcial: cada opción correcta que marcás suma. Por eso hay que conocer bien **qué se conecta dónde**, los números de la ESP32 y qué hace cada función. Después de leer esto, hacé el **banco de preguntas** (archivo aparte): ahí están todas las preguntas que ya salieron con la respuesta y la explicación.

1 IoT y sistemas embebidos

IoT (Internet de las cosas) es la interconexión de objetos cotidianos con internet para recolectar y compartir datos **sin intervención de personas**. Los dispositivos se comunican **M2M** (máquina a máquina) por cable, WiFi, Bluetooth, etc. Un dispositivo IoT es un objeto con conexión a internet e inteligencia de software que **mide** el mundo con **sensores** y **actúa** con **actuadores**. Ej.: Alexa, luces, enchufes, termostatos, cámaras.

El componente esencial es el **microcontrolador**: un **sistema embebido** de control, diseñado para una **función dedicada**, normalmente en tiempo real (a diferencia de una PC, de propósito general).

Recursos limitados

Poca CPU, RAM y Flash para bajar costo y consumo → a veces se omiten capas de seguridad.

Ciclos de vida largos

Funcionan años sin actualizarse → acumulan *deuda de seguridad*.

Exposición física

Un atacante puede tocar el hardware → ataques a nivel físico.

Ventajas del IoT: conectividad, automatización y eficiencia, monitoreo remoto, productividad, personalización y decisiones basadas en datos.

Ciclo sensor → controlador → actuador (lazo cerrado)

Sensor mide → Controlador (ESP32) compara con lo deseado → Actuador corrige ↺

2 La ESP32 en números

Es un **SoC** (System on Chip) de **Espressif Systems**, de bajo costo y bajo consumo, con **WiFi y Bluetooth integrados**. Es “un Arduino con esteroides”.

Característica	ESP32 (DOIT DevKit V1)	Arduino UNO (para comparar)
Procesador	**Tensilica Xtensa LX6**, **doble núcleo**, **32 bits**, hasta **240 MHz**. Los núcleos trabajan por separado o juntos (procesamiento paralelo). Tiene DSP y acelerador de cifrado.	ATmega328P, 8 bits, **16 MHz**
Tensión de trabajo	**3,3 V** (por USB entran 5 V y el regulador los baja). Más de 3,3 V en un pin **puede dañarla**.	5 V
Memorias	**Flash** hasta 4 MB (programa, no volátil) · **SRAM** 520 KB (datos en ejecución, volátil) · **ROM** 448 KB (bootloader) · **EEPROM** (configuraciones que sobreviven al reinicio)	
Conectividad	WiFi 802.11 b/g/n en **2,4 GHz** · Bluetooth 4.2 clásico + **BLE**	No tiene
Bajo consumo	**Coprocesador ULP** (lee sensores y despierta al sistema), RTC, PMU. Deep sleep.	
USB	El chip **no tiene USB nativo**: la placa trae un **chip conversor USB–UART** (CP210x, por eso el driver).	

Periféricos (lo que más se pregunta)

Periférico	Cantidad / dato clave
GPIO	30 o 36 pines según la placa. Cada pin se programa para distintas funciones.
ADC (analógico → digital)	2 conversores de **12 bits** → valores **0 a 4095**. Con WiFi encendido, evitar los pines **ADC2**.
DAC (digital → analógico)	2 canales de **8 bits** (0 a 255) en **GPIO25 y GPIO26**: las únicas salidas analógicas "puras".
PWM	**16 canales**; frecuencia y resolución configurables **por canal**.
UART · I2C · SPI · I2S	3 UART · 2 I2C · 3 SPI · 2 I2S (hardware dedicado: el procesador delega la comunicación).
Touch	9–10 pines capacitivos: botones táctiles sin partes mecánicas; pueden despertar del deep sleep.
Botones	**EN (ENABLE)** = reset. **BOOT** = modo de carga de firmware.
LED integrado	En el **GPIO 2** (`LED_BUILTIN`). Se maneja con `digitalWrite()`; sirve para pruebas rápidas y depuración.
Pines solo entrada	**GPIO34 a GPIO39**: solo entrada (no sirven como salida ni para PWM).

Placa de la cátedra (placa UTN)

Potenciómetro en el **pin 32** (entrada analógica), sensor **DHT22** (temperatura y humedad, salida **digital**) en el **pin 33**, display **OLED por I2C** (librería **Adafruit SH110X**). Los videos de Pedroni y Laboret usan la **SSD1306**: el código es casi igual, pero hay que cambiar la librería y el objeto del display. El profe recomienda alimentar siempre por USB y no usar VIN.

3 Señales: digital, analógica, PWM y DAC

Digital (2 estados)

HIGH ≈ 3,3 V (1 lógico) · **LOW** ≈ 0 V (0 lógico, apaga una salida).
Una entrada sin conectar queda **flotante** → se usa **pull-up** (por defecto HIGH) o **pull-down** (por defecto LOW): `pinMode(pin, INPUT_PULLUP)`.

Analógica → ADC

Tensión continua entre 0 y 3,3 V. `analogRead(pin)` devuelve **0 a 4095** (12 bits). Resolución ≈ 3,3 V / 4096 ≈ **0,8 mV**. N bits → 2^N valores (8 bits: 256 · 10 bits: 1024 · 12 bits: 4096).

PWM (no es analógica)

Onda cuadrada de frecuencia fija y **ancho de pulso variable**. **Duty cycle** = t_{alto} / T: 0 % siempre LOW, 100 % siempre HIGH. Ej.: con 5 V, 20 % de duty → 1 V promedio.
`analogWrite(pin, 0…255)` con 8 bits.

DAC (sí es analógica)

Genera una **salida analógica continua** (GPIO25/26, 8 bits). Sirve para controlar dispositivos analógicos.

¿Qué se conecta dónde? (sale siempre)

Tipo de pin	Se conecta	Función
Entrada digital	Pulsador, interruptor/llave, sensores discretos (fin de carrera), touch	`digitalRead()`
Entrada analógica	**Potenciómetro**, sensor con salida de tensión variable (temperatura, LDR)	`analogRead()` → 0–4095
Salida digital	**LEDs, relés, pantallas LCD**, transistores	`digitalWrite()`
Salida PWM	**LED** (brillo), **servo** (posición), **buzzer** (tono), motor DC (velocidad, con driver)	`analogWrite()` / canales LEDC
Sensor digital	DHT22, sensores I2C/SPI	No necesita ADC; viene “procesado” y es más robusto al ruido

Cuidado eléctrico

Más de 3,3 V en un pin → puede **dañar el microcontrolador** (usar divisor de tensión o conversor de nivel). Un GPIO da poca corriente: un motor o una tira de LEDs se maneja con un **transistor o un relé**, nunca directo.

4 Programación (C++ para Arduino)

C/C++ (Ritchie, AT&T, 1969–73; C++ de Stroustrup): **compilado**, **fuertemente tipado**, estructurado + POO, memoria a cargo del programador.

Un **sketch** tiene dos funciones:

- `setup()`: corre **una sola vez** al iniciar (configurar pines, Serial, WiFi).
- `loop()`: se repite **para siempre**.

Función	Qué hace
`pinMode(p, m)`	INPUT, OUTPUT, INPUT_PULLUP, INPUT_PULLDOWN
`Serial.begin(115200)`	Abre el puerto serie (UART) a esa velocidad
`Serial.available()`	Cuántos bytes llegaron para leer
`map(x, a1, a2, b1, b2)`	**Escala** un rango a otro (0–4095 → 0–255)
`millis()`	ms desde que arrancó (en `unsigned long`)
`delay(ms)`	**Frena todo** el programa ese tiempo

millis() vs delay() (pregunta fija)

`delay()` **bloquea**: mientras espera no lee sensores ni atiende WiFi; abusar de él afecta el rendimiento. `millis()` permite hacer **temporizadores sin bloquear** el loop: se guarda el tiempo y se compara.

Tipos de datos (como los da la cátedra): bool/byte/char 8 bits · int, word, short 16 bits · long, unsigned long, float 32 bits. **Ojo:** en la ESP32 el `int` ocupa **32 bits** (los 16 bits son del Arduino UNO); si en el parcial aparece 16, es porque lo toman del apunte de la cátedra. `millis()` se guarda en `unsigned long`. El punto flotante no es nativo: evitarlo si se puede. **Interrupciones**: el micro responde al instante a un evento (ISR); deben ser cortas y no bloqueantes.

5 Protocolos de comunicación

Síncrona: emisor y receptor comparten una señal de **reloj** (SPI, I2C) → rápida y continua. **Asíncrona**: sin reloj; cada dato lleva **bits de inicio y parada** (UART) → cableado simple, punto a punto.

	UART (serie)	I2C	SPI
Tipo	Asíncrona	Síncrona	Síncrona
Líneas	**TX** y **RX** (cruzadas: TX→RX)	**2**: **SDA** (datos) y **SCL** (reloj)	**4**: **MOSI**, **MISO**, **SCLK** y **CS/SS**
Topología	Punto a punto	Bus maestro–esclavo, **dirección de 7 bits** → muchos dispositivos con 2 cables	Maestro–esclavo, CS elige el esclavo
Velocidad	La fija el **baudrate** (igual en ambos)	Baja–media	**Alta**
Ejemplos	Monitor serie, GPS, depuración (RS232)	OLED, sensores (BME280), RTC, expansores. Librería `Wire`	Tarjeta SD, pantallas gráficas

6 Redes y la nube

WiFi

Hace falta el **SSID** (nombre de la red) y el **password**. Librería `WiFi.h` (viene instalada):

```
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
delay(500);
Serial.println(WiFi.localIP());
```

Modos: **STA** (se conecta al router, como un celular), **AP** (crea su propia red) y **mixto**.

Servidor web en la ESP32

`WiFiServer server(80);` El navegador (cliente) manda un **GET** (ej. `http://192.168.1.11/ledon`), la ESP32 lo procesa, prende el LED y responde con **HTML**. Cliente y placa en la **misma red** (o IP fija para internet).

Protocolo	Idea clave
TCP	Orientado a conexión, entrega ordenada y sin errores, pero usa más memoria/CPU.
UDP	Sin conexión, "dispara y olvida": rápido y liviano, ideal para telemetría donde perder un paquete no importa.
HTTP / REST	La ESP32 como cliente de APIs o como servidor que expone sus datos.
WebSockets	Una conexión TCP persistente, bidireccional, en tiempo real (paneles de control).
MQTT	Liviano, modelo **publicador/suscriptor**: los dispositivos **publican** en un **topic** y otros se **suscriben**; un **broker** (ej. **Mosquitto**) intermedia. Puerto **1883** (8883 con TLS). Librería `PubSubClient`. Ideal para conexiones inestables y miles de dispositivos.
Bluetooth	**Clásico (BR/EDR)**: flujo continuo de datos (audio); red piconet maestro–esclavos; dirección de 48 bits. **BLE**: ultra bajo consumo; roles **central** (celular) y **periférico** (sensor); datos organizados en **GATT** (servicios y características).

Plataformas IoT de los prácticos

ThingSpeak

De MATLAB. **Canales** con **8 campos** de datos (+3 de ubicación y 1 de estado). Se envían datos por **HTTP** con la **API key**. Gráficos y análisis con MATLAB. Registro con mail institucional.

Arduino IoT Cloud

Dashboard web para controlar la placa (ej. prender el LED). Librerías `ArduinoIoTCloud` y `Arduino_ConnectionHandler`; la placa se identifica con su **Device Key**.

Bot de Telegram

Se crea con **@BotFather** (`/newbot`), que da el **TOKEN**. Librería `UniversalTelegramBot`. Se manda la orden desde el celular.

Una plataforma IoT adquiere datos, los analiza localmente (edge) o en la nube, los almacena, ordena acciones a las "cosas" y presenta la información, de forma segura y escalable. Otras: Azure IoT, AWS IoT, Google Cloud IoT.

Seguridad: la ESP32 trae aceleradores de cifrado por hardware (AES, SHA, RSA, generador de números aleatorios) para TLS/SSL, **arranque seguro** (secure boot) y cifrado del firmware.

7 Los sketches que tenés que saber leer

Son los patrones de los prácticos. En el parcial te pueden preguntar qué hace una función o qué pasa si cambiás algo.

A · Blink (salida digital)

```
const int LED = 2;         // LED integrado
void setup() {
  pinMode(LED, OUTPUT);
}
void loop() {
  digitalWrite(LED, HIGH); // 3,3 V: prende
  delay(500);
  digitalWrite(LED, LOW);  // 0 V: apaga
  delay(500);
}
```

B · Pulsador con pull-up (entrada digital)

```
const int BOTON = 4;
void setup() {
  pinMode(BOTON, INPUT_PULLUP); // suelto=HIGH
  Serial.begin(115200);
}
void loop() {
  if (digitalRead(BOTON) == LOW) // apretado
    Serial.println("Boton presionado");
}
```

C · Potenciómetro → brillo del LED (ADC + map + PWM)

```
const int POT = 32, LED = 26;
void setup() { pinMode(LED, OUTPUT); }
void loop() {
  int lectura = analogRead(POT);     // 0..4095
  int brillo = map(lectura,0,4095,0,255);
  analogWrite(LED, brillo);  // duty 0..100 %
  float volt = lectura * 3.3 / 4095;
}
```

D · Temporizador sin bloquear (millis)

```
unsigned long antes = 0;
void loop() {
  if (millis() - antes >= 1000) {  // paso 1 s?
    antes = millis();
    // leer sensor, enviar dato...
  }
  // el loop sigue libre para otras tareas
}
```

E · Comando por el monitor serie

```
void loop() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd == "LUZ_ON")  digitalWrite(RELAY, HIGH);
    if (cmd == "LUZ_OFF") digitalWrite(RELAY, LOW);
  }
}
```

F · MQTT (publicar y suscribirse)

```
#include <WiFi.h>
#include <PubSubClient.h>
WiFiClient esp;  PubSubClient client(esp);
void setup() {
  WiFi.begin(ssid, password);        // 1) WiFi
  client.setServer(broker, 1883);    // 2) broker
  client.setCallback(callback);      // 3) recibir
}
void loop() {
  if (!client.connected()) {
    client.connect("esp-a");
    client.subscribe("casa/luz");  // escucha
  }
  client.loop();
  client.publish("casa/temp","23.5"); // publica
}
```